

UNIVERSIDAD DE CUNDINAMARCA

Sede Fusagasugá

Programa académico: Ingeniería de Sistemas y Computación

Asignatura: Ingeniería de Software I

FusaRoute

Sistema de información de transporte público para Fusagasugá y la región del Sumapaz

Documento V2

Integrantes del equipo

Santiago Bermúdez Gómez

Angélica María Aranguren Rozo

Docente: Ing. Luiferney Ortiz Parra

Semestre académico: 2026-2 (quinto semestre)

Fecha de entrega: 28 de septiembre de 2026

Tabla de contenido

Registro de cambios de esta versión

1. Introducción del proyecto
2. Descripción general del sistema
3. Enfoque metodológico del proyecto
4. Identificación y clasificación de requerimientos
5. Dependencias del proyecto
6. Configuración tecnológica del proyecto
7. Riesgos del proyecto
8. Diseño de base de datos
9. Arquitectura del sistema
10. Historias de usuario y casos de uso
11. Contrato de la API
12. Métricas de calidad
13. Estado de avance, problemas y próximos pasos
14. Conclusiones del proyecto

Referencias

Nota: las secciones 1 a 8 siguen la estructura pedida por el docente; las secciones 9 a 13 se agregaron en esta versión y la de conclusiones pasó de la 9 a la 14 por ese motivo.

Registro de cambios de esta versión

Este documento no parte de un Documento V1 con la estructura de nueve secciones: en el repositorio de la asignatura solo existe la estructura que pide el docente (Entregable Documento V1) y la Actividad 3 en su versión 3. Este V2 toma como base esa Actividad 3 y todos los cambios aprobados desde entonces, y los reorganiza en la estructura completa del docente. Cada cambio queda registrado con su razón, porque ante el comité cambiar de decisión con argumentos suma y cambiar sin registrar el porqué resta.

Fecha	Cambio	Razón
24/08/2026	Actividad 3 v3: 5 métricas de calidad (una por atributo ISO/IEC 25010), cada una con instrumento ejecutable.	Una revisión interna encontró que 7 de las 8 métricas de la v2 estaban mal o medianamente definidas y 1 era redundante.
14/09/2026	RNF-02 rediseñado: frontend en Vercel y backend en Render (tier gratis), sin promesa de servidor 24/7.	El tier gratis de Render se duerme a los 15 minutos y tarda 30 a 60 s en reactivarse; prometer un RTO de servidor siempre activo era falso.
15/09/2026	Se agregó RF-15 (catálogo público de rutas) y el criterio de última milla en RF-04. RNF-01 recibió un tope de 1 MB gzip para el GeoJSON.	El catálogo completo solo lo veía el administrador, lo que contradice la justificación del proyecto (información pública clara).
21/09/2026	Sprint 2 reestructurado para el comité; SCRUM-13 y SCRUM-19 se partieron (SCRUM-155 y SCRUM-154). Los tres ambientes corren sobre Supabase y el esquema se versiona con Flyway.	El Sprint 1 cerró con velocidad 0 y no existía ninguna funcionalidad implementada en main.
27/09/2026	RNF-05 corregido: cero vulnerabilidades altas o críticas sin suprimir, con las supresiones justificadas y fechadas.	La cifra "0 detectadas" era inalcanzable: hay CVE de Spring sin versión corregida publicada en abierto.
28/09/2026	Este V2: estructura completa del docente; estado real de implementación; arquitectura, modelo de datos y contrato de API tomados del código; RF-03 adelantado al Sprint 2 (implementado en rama, sin merge).	Documentar el sistema tal como está, separando lo implementado de lo planificado.

1. Introducción del proyecto

En Fusagasugá circulan más de veinte rutas urbanas de busetas y no existe información pública clara sobre ellas: ni siquiera Google Maps las tiene registradas. Un habitante o un turista que quiere llegar a un destino puede hacerlo por varias rutas, y sin información pierde tiempo y dinero eligiendo. La necesidad la respalda una encuesta propia del equipo (2025) y una población regional estimada en unos 285.000 habitantes (DANE).

FusaRoute es un sistema de información web: un backend que expone una API REST y un frontend de una sola página que la consume. Se usa desde el celular o el computador, con internet, dentro de Fusagasugá y las rutas que la conectan con Chinauta, Pasca y Arbeláez.

El objetivo general es ofrecer información clara y confiable del transporte público en buseta: qué rutas existen, por dónde pasan, cuánto cuestan y cuál conviene tomar entre un origen y un destino. Este semestre el equipo entrega el primer incremento funcional (cuenta de usuario y catálogo público de rutas) sobre una arquitectura hexagonal pensada para crecer.

2. Descripción general del sistema

2.1 Problema identificado

La información de las rutas de buseta existe solo en la práctica de quienes las usan a diario. No hay un lugar público donde consultar qué rutas hay, qué barrios recorren ni cuánto cuestan, y las rutas no aparecen en los mapas comerciales. Esto afecta más a quienes llegan por primera vez a la ciudad y a quienes deben desplazarse a un barrio que no conocen.

2.2 Objetivo del sistema

Reunir en un solo lugar las rutas de buseta de Fusagasugá y la región, y ayudar al usuario a decidir cuál tomar: mostrar las rutas, sus barrios y tarifas y, en fases siguientes, sugerir la de menor tiempo estimado entre dos puntos.

2.3 Usuarios del sistema

Tipo de usuario	Descripción
Visitante	Cualquier persona, sin cuenta. Explora el catálogo público de rutas (RF-15).
Usuario Final	Persona registrada. Además de lo público, busca rutas, guarda historial y destino favorito, y deja comentarios.
Administrador	Gestiona las rutas (alta, modificación, suspensión temporal), responde comentarios y consulta las métricas de uso. El rol existe en la base de datos (role = USER o ADMIN); sus funciones se implementan en incrementos posteriores.

2.4 Alcance del proyecto

Alcance geográfico: las rutas urbanas de Fusagasugá, la ruta a Chinauta y las intermunicipales Fusagasugá–Pasca y Fusagasugá–Arbeláez. Regla de frontera: toda ruta incluida debe tocar

Fusagasugá. Silvania quedó fuera este semestre por ser menos realista para la primera vez del equipo integrando mapas.

Sí incluye (semestre)	No incluye
Registro e inicio de sesión del Usuario Final; ver y editar su perfil	GPS en tiempo real de las busetas (descartado por riesgo de alcance)
Catálogo público de rutas, con barrios y tarifa	Pagos en línea
Búsqueda de ruta origen–destino, con barrios, tarifa y punto de abordaje más cercano	Notificaciones push
Aviso de congestión histórica y destino favorito	Rutas que no toquen Fusagasugá; el resto del Sumapaz (Silvania, Granada, Tibacuy, Venecia)
Historial de búsquedas y caja de comentarios	Recuperación de contraseña y tarifas diferenciales (estudiante, adulto mayor)
Modo offline básico por GeoJSON (sin tiempo de llegada)	
Administrador: CRUD de rutas, suspensión temporal con notificación, respuesta a comentarios y 5 métricas de uso	

Lo que existe hoy de ese alcance está detallado en la sección 13. En resumen: registro, inicio de sesión (sin el bloqueo por intentos fallidos), catálogo público (sin mapa ni aviso de congestión) y perfil (en rama, sin merge).

3. Enfoque metodológico del proyecto

3.1 Análisis del contexto del proyecto

- **¿Los requerimientos pueden cambiar?** Sí, y ya cambiaron: el modo offline, las tarifas, el aviso de congestión, el destino favorito y las notificaciones se redefinieron más de una vez, y el 15/09 apareció un requisito nuevo (RF-15).
- **¿Hay incertidumbre técnica?** Alta. Es la primera vez del equipo con mapas, con Spring Boot y con Angular; además hay dependencias externas (Google Maps, planes gratis de Render y Supabase).
- **Riesgos iniciales:** falta de experiencia en el stack, dificultad para conseguir los mapas oficiales de las empresas de transporte y la disponibilidad real de dos personas con otras materias (sección 7).
- **¿El cliente podría modificar prioridades?** Sí. El docente actúa como comité y redefine qué se entrega en cada sustentación; el Sprint 2 se reestructuró por ese motivo.

3.2 Enfoque metodológico seleccionado

Enfoque iterativo–incremental con Scrum como marco de trabajo. El enfoque dice cómo crece el producto (por incrementos demostrables, cada uno usable de punta a punta) y Scrum dice cómo se organiza el equipo para producirlos. La base tradicional es de linaje RUP —arquitectura

definida temprano, que es la hexagonal ya comprometida— y la evolución hacia ágil se justifica por la incertidumbre real descrita arriba.

- **Ventajas:** cada incremento se puede demostrar y corregir antes de seguir; la arquitectura se fija temprano y el alcance funcional se ajusta.
- **Qué ayuda a evitar:** descubrir al final que lo construido no responde a lo que pide el docente, y planear grande sobre información que todavía no se tiene.

3.3 Uso de Scrum como marco de trabajo

Evento	Cuándo	Cómo se adopta
Sprint Planning	Martes 18:00	Al salir de la clase de 16–18.
Daily Scrum	Diario	Se sustituyó por un check-in escrito (ayer / hoy / bloqueos): dos personas que comparten tres clases semanales no sostienen una reunión diaria. Se descartó por escrito para no prometer un evento que no se cumple.
Sesión de pareja	Jueves 13:00 (1 h)	Al salir de la clase de 11–13.
Sprint Review y Retrospectiva	Lunes en la noche	Cierre de sprint; el informe queda en Jira.

Duración de los sprints: una semana, de martes a lunes, de modo que a cada sustentación quincenal lleguen dos sprints cerrados. Las fechas de comité no siguen un patrón fijo: se le preguntan al docente y no se suponen. Backlog: Jira, proyecto único SCRUM, con 5 épicas que son a la vez los 5 incrementos demostrables (cuenta de usuario; búsqueda de ruta; historial y favoritos; comentarios; administración), 15 historias de usuario y 7 requisitos no funcionales.

Persona	Capa de proceso (Scrum)	Capa técnica (docente)
Angélica	Product Owner	Responsable de Frontend y de Documentación
Santiago	Scrum Master	Líder Técnico y Responsable de Backend
Ambos	Development Team	Responsable de Pruebas (compartido)

"Responsable" significa quien responde ante el comité y revisa los PR de esa área, no quien la programa solo: ambos programan frontend y backend. La capa técnica debe rotar cada 2 sprints; esa rotación todavía no ha ocurrido y se registrará al cierre del Sprint 3.

4. Identificación y clasificación de requerimientos

4.1 Requerimientos funcionales

El docente sugiere entre 8 y 12 requerimientos funcionales. El backlog aprobado por el equipo el 15/09/2026 tiene 15, agrupados en 5 incrementos; se presentan todos porque son los que respaldan el alcance declarado. Cada uno se redactó con el formato "El sistema debe...".

Código	Nombre	Descripción	Tipo de usuario
RF-01	Registro de usuario	El sistema debe permitir registrarse con nombre, correo y contraseña.	Usuario Final
RF-02	Inicio de sesión	El sistema debe permitir iniciar sesión con correo y contraseña y emitir un token con validez de una semana.	Usuario Final
RF-03	Datos personales	El sistema debe permitir ver y editar los datos personales.	Usuario Final
RF-04	Mejor ruta	El sistema debe sugerir la mejor ruta de busetas entre un origen y un destino marcados por el usuario, indicando el punto de abordaje más cercano y la distancia a pie.	Usuario Final
RF-05	Barrios de la ruta	El sistema debe mostrar los barrios o comunas por los que pasa la ruta sugerida.	Usuario Final
RF-06	Costo del pasaje	El sistema debe mostrar el costo del pasaje de la ruta sugerida.	Usuario Final
RF-07	Historial	El sistema debe guardar y mostrar las últimas búsquedas del usuario, asociadas a su cuenta.	Usuario Final
RF-08	Caja de comentarios	El sistema debe permitir dejar un comentario o sugerencia, editable mientras no haya sido respondido.	Usuario Final
RF-09	Aviso de congestión	El sistema debe avisar si el trayecto sugerido suele estar congestionado a la hora de la consulta.	Usuario Final
RF-10	Destino favorito	El sistema debe permitir marcar un destino como favorito y precargarlo con su aviso de congestión al abrir la búsqueda.	Usuario Final
RF-11	CRUD de rutas	El sistema debe permitir al administrador crear, consultar, modificar y eliminar (de forma lógica) rutas.	Administrador
RF-12	Suspensión temporal	El sistema debe permitir suspender una ruta temporalmente y notificar dentro de la app a quienes la usan seguido.	Administrador
RF-13	Métricas de uso	El sistema debe mostrar al administrador cinco métricas de uso de la aplicación.	Administrador
RF-14	Responder comentarios	El sistema debe permitir al administrador leer y responder los comentarios de los usuarios.	Administrador
RF-15	Catálogo público	El sistema debe permitir a cualquier visitante, sin cuenta, explorar el listado de rutas activas y el detalle de cada una.	Visitante

4.2 Requerimientos no funcionales

Categoría (ISO/IEC 25010)	Descripción	Criterio de validación
RNF-01 Rendimiento	El sistema debe responder las consultas de ruta con un p95 menor a 8 s extremo a extremo en red 4G típica; caché de 5 min por par origen–destino; el bundle GeoJSON de la red completa pesa ≤ 1 MB gzip; modo offline básico sin tiempo de llegada.	Spring Boot Actuator + Micrometer (p95), Lighthouse en CI (LCP $< 2,5$ s), tamaño del bundle verificado en el build.
RNF-02 Disponibilidad	El backend (Render, tier gratis) y el frontend (Vercel) deben responder correctamente en al menos el 95 % de los latidos en horario hábil (lun–vie 7:00–21:00), incluyendo el cold start; RPO < 24 h. No se compromete un RTO de servidor siempre activo.	UptimeRobot (latido cada 5 min) + backup diario de PostgreSQL y restore probado al cierre de cada sprint.

Categoría (ISO/IEC 25010)	Descripción	Criterio de validación
RNF-03 Mantenibilidad	El backend debe mantener la separación estricta de capas del hexágono, con la lógica de negocio sin depender de frameworks: 0 violaciones y al menos 70 % de cobertura.	ArchUnit en CI + JaCoCo con umbral bloqueante.
RNF-04 Usabilidad	La búsqueda de ruta debe completarse en máximo 4 pasos y el login pedirse como máximo una vez por semana; tasa de éxito $\geq 80\%$ al primer intento.	Prueba con al menos 5 usuarios externos al equipo antes de cada sustentación.
RNF-05 Seguridad	El sistema no debe tener vulnerabilidades altas o críticas ($CVSS \geq 7$) sin suprimir; toda supresión lleva justificación de inalcanzabilidad, razón de no actualizar y fecha de revisión.	OWASP Dependency-Check (failBuildOnCVSS=7) + análisis estático en CI, en cada Pull Request.
RNF-06 Seguridad (contraseñas)	Las contraseñas deben almacenarse con hash bcrypt (factor de costo ≥ 10), nunca en texto plano.	Test de integración: el hash guardado no es igual a la contraseña y el login funciona con la original.
RNF-07 Compatibilidad	La aplicación debe funcionar en Chrome (Android), Safari (iOS) y un navegador de escritorio.	Prueba manual en dispositivos físicos y emulador, antes de cada sustentación.

4.3 Priorización de requerimientos

Prioridad	Requerimientos
Alta	RF-01, RF-02, RF-04, RF-05, RF-06, RF-09, RF-11, RF-12, RF-15
Media	RF-03, RF-07, RF-08, RF-10, RF-13, RF-14
Baja	Ninguno: lo que iba a ser de baja prioridad se sacó del alcance (pagos, push, GPS en tiempo real).

Criterio: es de prioridad alta lo que, si falta, el sistema no cumple su propósito de informar y ayudar a elegir ruta (cuenta, búsqueda, barrios, costo, congestión), lo que el administrador necesita para que esa información sea vigente (CRUD y suspensión) y el catálogo público, que es la razón de ser del proyecto. Es de prioridad media lo que mejora la experiencia o la operación pero no impide usar el sistema. La prioridad de RF-15 (Alta) la asigna este documento porque el requisito se creó después de la Actividad 3; se somete a la confirmación de la Product Owner.

5. Dependencias del proyecto

Dependencia	Descripción	Impacto
Base de datos en Supabase	PostgreSQL administrado, un proyecto por ambiente (DEV, PRE, PROD). El tier gratis permite 2 proyectos por cuenta, así que el tercero va en una segunda cuenta.	Alto
Google Maps (Directions API)	Cronometrar las rutas candidatas para RF-04. Sin integrar todavía; es un puerto de salida futuro (TravelTimeProvider).	Alto
Render y Vercel (tier gratis)	Despliegue de backend y frontend (Sprint 3). Render se duerme a los 15 min de inactividad.	Alto

Dependencia	Descripción	Impacto
Datos de las rutas	Mapas oficiales y trazados de las empresas de transporte; posiblemente haya que trazar las rutas a mano. Hoy hay 8 rutas sembradas con barrios y tarifas, sin trazado GeoJSON.	Alto
Tiempo del equipo	Dos personas con otras cinco materias; el Sprint 1 cerró con velocidad 0.	Alto
CI en GitHub Actions y clave de la NVD	Los gates de calidad y seguridad corren en CI; OWASP Dependency-Check necesita una NVD_API_KEY gratuita.	Medio
Jira y GitHub	Backlog, sprints y trazabilidad de rama, commit y PR por la key SCRUM-N.	Medio

6. Configuración tecnológica del proyecto

Componente	Tecnología
Frontend	Angular 21 · TypeScript 5.9 · Node 22 · pruebas con Vitest
Backend	Java 25 · Spring Boot 3.5.16 · arquitectura hexagonal (puertos y adaptadores)
Autenticación	Spring Security + JWT (propio del backend, no Supabase Auth) · hash BCrypt
Base de datos	PostgreSQL 17 en Supabase · esquema versionado con Flyway (V1 a V4) · Hibernate solo valida (ddl-auto=validate)
Calidad y seguridad	ArchUnit · JaCoCo (umbral 0,70) · OWASP Dependency-Check · Checkstyle
Integración continua	GitHub Actions (mvn -B verify en el backend)
Control de versiones	Git y GitHub: tres repositorios (material del curso, backend, frontend), trunk-based con Pull Request obligatorio
Gestión del proyecto	Jira (proyecto SCRUM)
Despliegue (planificado, Sprint 3)	Frontend en Vercel · backend en Render (tier gratis)
Mapas (planificado, Sprint 4 en adelante)	Google Maps Directions API, solo para cronometrar rutas

Regla de versiones: el proyecto arranca sobre la versión vigente de cada herramienta. Spring Boot se queda en la última 3.5 y no salta a la línea 4 porque es la primera vez del equipo con Spring y el material para resolver errores es de 3.x. Cualquier salto de versión mayor es una decisión propia con su tarjeta. El estado real de las dependencias está en la evidencia de dependencias.

7. Riesgos del proyecto

Riesgo	Descripción	Nivel	Estado a 28/09/2026
Primera vez con mapas	El equipo nunca integró mapas en una aplicación web.	Alto	Sin integrar; se aborda desde el Sprint 4.
Primera vez con Spring Boot y Angular	El equipo puede no dominar el stack.	Alto	Materializado en parte: el Sprint 1 cerró con velocidad 0.
Información de rutas	Puede ser difícil conseguir los mapas oficiales de las empresas.	Alto	8 rutas sembradas; los trazados GeoJSON están pendientes (path_geojson es NULL).

Riesgo	Descripción	Nivel	Estado a 28/09/2026
Carga del equipo	Dos personas, otras cinco materias, 17 puntos apostados en un sprint sin historial.	Alto	El Sprint 2 cerró en 17 puntos según Jira; una sola medición no es capacidad.
Entorno de producción inexistente	PROD no está desplegado; el backend corre desde un portátil.	Alto	Se despliega en el Sprint 3.
Cold start de Render	La primera petición tras 15 min de inactividad tarda 30 a 60 s y puede empujar el p95 por encima de 8 s.	Medio	Sin resolver; decidir antes de la sustentación si se acepta o se paga un plan.
Vulnerabilidades sin corrección abierta	Hay CVE de Spring de CVSS ≥ 7 sin versión corregida publicada en abierto.	Medio	2 bloques de supresión con 15 CVE, todos con fecha de revisión 15/01/2027.
Cambios de requisitos	El comité puede redefinir prioridades.	Medio	Materializado: RNF-01 y RNF-02, RF-10, RF-12 y RF-13 se redefinieron; se agregó RF-15.

Los niveles son una evaluación del equipo a la fecha del documento.

8. Diseño de base de datos

El esquema lo crea y versiona Flyway (db/migration, migraciones V1 a V4); Hibernate solo lo valida. El modelo refleja el código real: cuatro tablas.

8.1 Modelo entidad–relación

Modelo entidad-relación (esquema Flyway V1-V4)

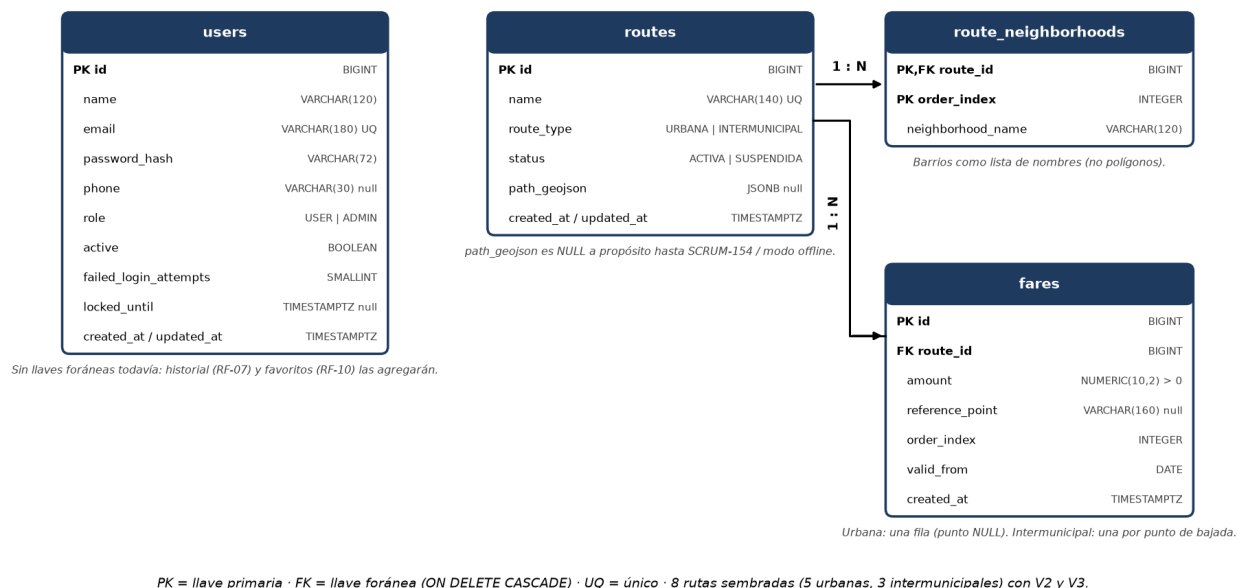


Figura 1. Modelo entidad–relación del esquema V1–V4.

8.2 Modelo relacional

Tabla	Descripción
users	Usuario final o administrador. Autenticación propia; la contraseña se guarda solo como hash BCrypt.
routes	Ruta de buseta: recorrido directo sin transbordos. Ida y vuelta son dos registros distintos. Estado ACTIVA o SUSPENDIDA.
route_neighborhoods	Barrios por los que pasa cada ruta, en orden de recorrido. Se guardan como lista de nombres, no como polígonos.
fares	Tarifa del pasaje. Urbana: una fila sin punto de referencia. Intermunicipal: una fila por punto de bajada. Lleva fecha de vigencia.

Relaciones: una ruta tiene muchos barrios y muchas tarifas (ambas con borrado en cascada). La tabla users no tiene llaves foráneas todavía: las tendrán cuando se implementen el historial (RF-07) y el destino favorito (RF-10). Las columnas failed_login_attempts y locked_until ya existen para el bloqueo por intentos fallidos, que se implementa en SCRUM-155.

8.3 Diccionario de datos

Tabla	Campo	Tipo de dato	Descripción
users	id	BIGINT	Identificador (llave primaria, autogenerado)
users	name	VARCHAR(120)	Nombre del usuario
users	email	VARCHAR(180)	Correo; único
users	password_hash	VARCHAR(72)	Hash BCrypt de la contraseña
users	phone	VARCHAR(30)	Teléfono (opcional)
users	role	VARCHAR(20)	USER o ADMIN (por defecto USER)
users	active	BOOLEAN	Cuenta activa
users	failed_login_attempts	SMALLINT	Contador de intentos fallidos (reservado para SCRUM-155)
users	locked_until	TIMESTAMPZ	Fin del bloqueo temporal (reservado para SCRUM-155)
users	created_at / updated_at	TIMESTAMPZ	Auditoría
routes	id	BIGINT	Identificador
routes	name	VARCHAR(140)	Nombre de la ruta; único
routes	route_type	VARCHAR(20)	URBANA o INTERMUNICIPAL
routes	status	VARCHAR(20)	ACTIVA o SUSPENDIDA
routes	path_geojson	JSONB	Trazado GeoJSON (LineString); NULL hasta que se carguen los trazados
routes	created_at / updated_at	TIMESTAMPZ	Auditoría
route_neighborhoods	route_id	BIGINT	Ruta a la que pertenece (llave foránea, parte de la llave primaria)
route_neighborhoods	order_index	INTEGER	Posición en el recorrido (≥ 0)
route_neighborhoods	neighborhood_name	VARCHAR(120)	Nombre del barrio; único por ruta
fares	id	BIGINT	Identificador

Tabla	Campo	Tipo de dato	Descripción
fares	route_id	BIGINT	Ruta (llave foránea)
fares	amount	NUMERIC(10,2)	Valor del pasaje; mayor que 0
fares	reference_point	VARCHAR(160)	Punto de bajada (NULL en rutas urbanas)
fares	order_index	INTEGER	Orden de la tarifa dentro de la ruta
fares	valid_from	DATE	Fecha desde la que rige la tarifa
fares	created_at	TIMESTAMPTZ	Auditoría

9. Arquitectura del sistema

FusaRoute — arquitectura (estado al 28 de septiembre de 2026)

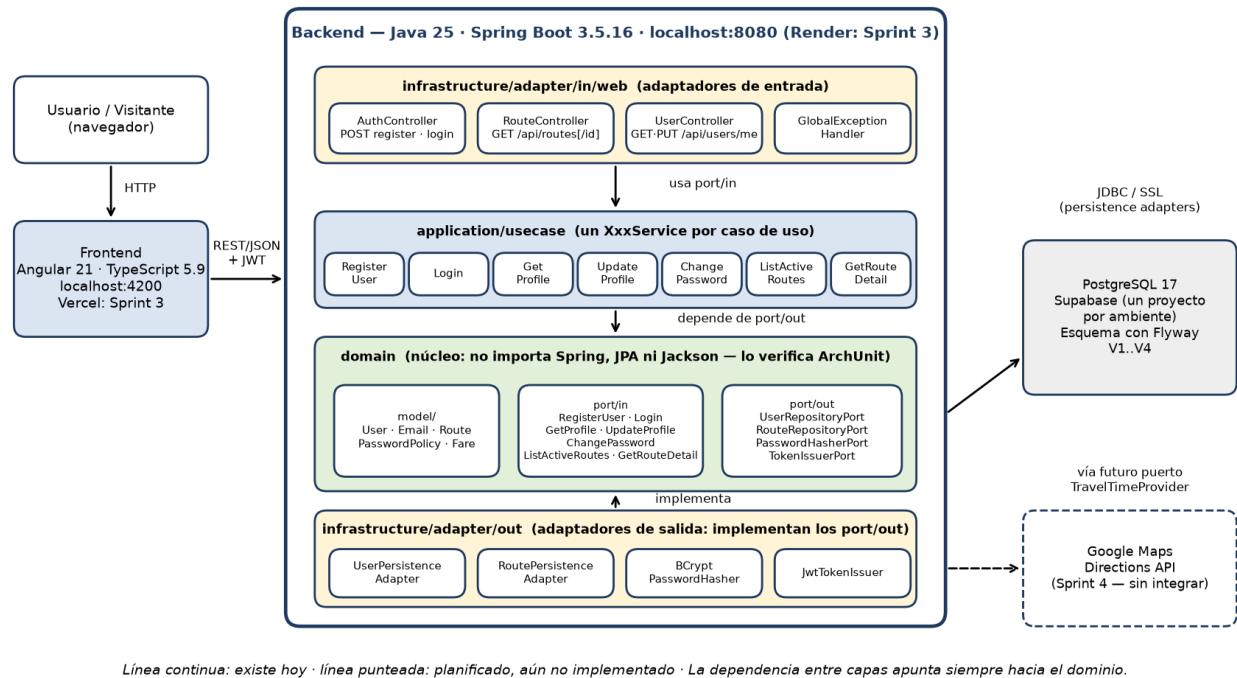


Figura 2. Arquitectura hexagonal del backend y su contexto (estado al 28/09/2026).

El backend sigue una arquitectura hexagonal: el dominio define las interfaces (puertos) y la infraestructura las implementa (adaptadores). La dependencia apunta siempre hacia el dominio y ArchUnit lo verifica en cada build (dos reglas: capas hacia adentro y dominio sin Spring, JPA ni Jackson).

La guía del docente prescribe Controller → Service → Repository. Hexagonal la contiene, con otros nombres y la dependencia invertida:

Guía del docente	FusaRoute (hexagonal)
Controller	infrastructure/adapters/in/web/

Guía del docente	FusaRoute (hexagonal)
Service	application/usecase/ y domain/model/
Repository	infrastructure/adapters/out/persistence/ (su interfaz vive en domain/port/out/)

Lo que hexagonal agrega es que la interfaz del repositorio pertenece al dominio: por eso se puede cambiar de base de datos sin tocar la lógica de negocio y por eso los casos de uso se prueban sin levantar Spring. La evidencia de los cinco principios SOLID, con archivo y línea, está en evidencia-solid.md.

Decisiones de arquitectura vigentes

Decisión	Estado
Backend Java 25 + Spring Boot 3.5, hexagonal; frontend Angular 21 + TypeScript 5.9	Vigente
PostgreSQL en Supabase, uno por ambiente; esquema con Flyway; Supabase solo como base administrada (sin Auth, Storage ni Realtime)	Vigente
Autenticación propia con Spring Security + JWT de una semana	Vigente
Cálculo de ruta por simulación en Google Maps con caché de 5 min	Vigente, sin implementar
Modo offline por GeoJSON con cálculo por distancia geométrica en el dominio, sin PostGIS	Vigente, sin implementar
Última milla: punto más cercano sobre el trazado y distancia a pie aproximada, sin ruteo peatonal de Google	Vigente, sin implementar
Despliegue: frontend en Vercel, backend en Render (tier gratis)	Decidido, se despliega en el Sprint 3

Cambio a sustentar ante el comité: el borrador inicial justificaba Supabase en parte por su módulo de login. Se decidió programar la autenticación en el backend porque el docente exige que ambos integrantes aprendan todo el stack, porque mantiene el dominio desacoplado de un proveedor (coherente con hexagonal) y porque da material real que defender en calidad y seguridad.

10. Historias de usuario y casos de uso

10.1 Historias de usuario

Las 15 historias fueron aprobadas por ambos integrantes el 15/09/2026 y están cargadas en Jira con su código HU_MFxx_xxx. La historia HU_MF02_001 se partió en dos (SCRUM-15 y SCRUM-153) por sobredimensionada.

Historia	RF	Jira	Estado a 28/09/2026
HU_MF01_001 Registro de usuarios	RF-01	SCRUM-12	Implementada (Done)
HU_MF01_002 Inicio de sesión	RF-02	SCRUM-13	Implementada, sin bloqueo por intentos (SCRUM-155, Sprint 3)
HU_MF01_003 Ver y editar datos personales	RF-03	SCRUM-14	Implementada en rama, sin merge; movida al Sprint 2

Historia	RF	Jira	Estado a 28/09/2026
HU_MF02_001 Mejor ruta por tiempo	RF-04	SCRUM-15	Pendiente
HU_MF02_001b Última milla y modo offline	RF-04	SCRUM-153	Pendiente (Sprint 4)
HU_MF02_002 Barrios de la ruta sugerida	RF-05	SCRUM-16	Pendiente (los barrios ya están en la base de datos)
HU_MF02_003 Costo del pasaje	RF-06	SCRUM-17	Pendiente (las tarifas ya están en la base de datos)
HU_MF02_004 Aviso de congestión	RF-09	SCRUM-18	Pendiente (Sprint 6)
HU_MF02_005 Catálogo público de rutas	RF-15	SCRUM-19	Implementada, sin mapa ni congestión (SCRUM-154, Sprint 6)
HU_MF03_001 Historial de búsquedas	RF-07	SCRUM-20	Pendiente
HU_MF03_002 Destino favorito	RF-10	SCRUM-21	Pendiente
HU_MF04_001 Caja de comentarios	RF-08	SCRUM-22	Pendiente
HU_MF04_002 Responder comentarios	RF-14	SCRUM-23	Pendiente
HU_MF05_001 CRUD de rutas	RF-11	SCRUM-24	Pendiente
HU_MF05_002 Suspensión temporal y notificación	RF-12	SCRUM-25	Pendiente
HU_MF05_003 Panel de métricas de uso	RF-13	SCRUM-26	Pendiente

Las cuatro historias del incremento de este sprint (HU_MF01_001, 002, 003 y HU_MF02_005) están desarrolladas sobre la plantilla del docente en docs/comite-2/HU/.

10.2 Casos de uso

Caso de uso	Historia	Actor
CU-001 Registro de usuario	HU_MF01_001	Usuario Final (sin sesión)
CU-002 Inicio de sesión	HU_MF01_002	Usuario Final (sin sesión)
CU-003 Ver y editar datos personales	HU_MF01_003	Usuario Final (con sesión)
CU-004 Catálogo público de rutas	HU_MF02_005	Visitante

Cada caso de uso, con su diagrama de caso de uso, su diagrama de flujo, la tabla de datos de entrada y salida y los requisitos no funcionales aplicables, está en docs/comite-2/CU/.

11. Contrato de la API

Los endpoints salen del código del backend (rama claude/gracious-clarke-2fqs2v, commit eba1429). La colección de Postman y un archivo .http de respaldo están en el repositorio del backend.

Método y ruta	Acceso	Solicitud	Respuesta
GET /health	Público	—	Estado del servicio
POST /api/auth/register	Público	name, email, password	201 con id, name, email, role y active del usuario creado
POST /api/auth/login	Público	email, password	200 con token, tokenType, expiresAt y los datos del usuario

Método y ruta	Acceso	Solicitud	Respuesta
GET /api/routes	Público	—	Lista de rutas activas con barrios y tarifas
GET /api/routes/{id}	Público	—	Detalle de una ruta activa
GET /api/users/me	JWT	—	Perfil: id, name, email, phone, role
PUT /api/users/me	JWT	name, email, phone	Perfil actualizado
PUT /api/users/me/password	JWT	currentPassword, newPassword	Cambio de contraseña (exige la actual)

Todo lo que no está en la lista de rutas públicas exige un token JWT en el encabezado Authorization: Bearer. Un token vencido o ausente devuelve un error JSON gestionado por el manejador de errores del backend.

12. Métricas de calidad

Una métrica por atributo ISO/IEC 25010, cada una con su instrumento. Se separa lo que ya está instrumentado de lo que todavía no se puede medir.

Atributo	Métrica comprometida	Instrumento	Estado a 28/09/2026
Rendimiento	p95 < 8 s en /api/routes/search; LCP < 2,5 s; bundle GeoJSON ≤ 1 MB gzip	Actuator + Micrometer; Lighthouse en CI	No medible: el endpoint de búsqueda no existe (Sprint 4).
Disponibilidad	≥ 95 % de latidos en horario hábil, con cold start; RPO < 24 h	UptimeRobot; backup diario de Supabase	No medible: PROD no está desplegado (Sprint 3).
Mantenibilidad	0 violaciones hexagonales; ≥ 70 % de cobertura	ArchUnit en CI; JaCoCo bloqueante	Instrumentado: 2 reglas ArchUnit y umbral 0,70 sobre domain y application en mvn verify. El porcentaje real no se midió al redactar este documento.
Usabilidad	≤ 4 acciones y éxito ≥ 80 % al primer intento	Prueba con ≥ 5 usuarios externos	No realizada.
Seguridad	0 vulnerabilidades CVSS ≥ 7 sin suprimir, con las supresiones declaradas	OWASP Dependency-Check (failBuildOnCVSS=7) + análisis estático	2 bloques de supresión que cubren 15 CVE, todos con revisión el 15/01/2027. npm audit del frontend: 0 vulnerabilidades.

Nota sobre RNF-05: el texto del requisito habla de 14 hallazgos sin versión corregida y el archivo de supresiones lista 15 identificadores CVE; la diferencia no está conciliada y se revisa en el próximo sprint.

13. Estado de avance, problemas y próximos pasos

13.1 Avance por sprint

Sprint	Fechas	Resultado
Preparación	09 al 14/09	Semana de preparación: no es un sprint y no genera informe.
Sprint 1	15 al 21/09	2 issues (6 puntos), 0 completados: velocidad 0. El trabajo técnico quedó en ramas sin mergear y la infraestructura no estaba registrada como issues.

Sprint	Fechas	Resultado
Sprint 2	22 al 28/09	17 de 17 puntos en Done según Jira (registro, login, catálogo, RNF-03, RNF-05, RNF-06) y 8 tareas de infraestructura cerradas. El comité es el lunes 28/09 a las 2:00 pm.

13.2 Problemas encontrados

- El Sprint 1 cerró vacío sin que nadie lo notara porque la infraestructura no figuraba en Jira; se corrigió creando 9 tareas visibles.
- Cuatro subtareas figuraban como terminadas sin estarlo; se devolvieron a To Do. Propuesta para el Sprint 3, pendiente de acordar entre ambos: cerrar un issue solo con su PR mergeado y el CI en verde.
- El umbral de JaCoCo (70 %) hacía fallar el propio build porque no había pruebas reales; no se bajó el número, se escribieron las pruebas (SCRUM-163).
- El archivo de supresiones OWASP tuvo un error de sintaxis XML durante cuatro días y el job de seguridad estuvo en rojo sin que la causa fuera evidente; por eso ahora el XML se valida en un paso propio antes del escaneo.
- El frontend necesita legacy-peer-deps=true por un desfase de parche entre @angular/service-worker y @angular/core; es deuda declarada.

13.3 Próximos pasos

- Mergear el PR de RF-03 (SCRUM-14) a main del backend y del frontend.
- Sprint 3 (29/09 al 05/10): despliegue real en Vercel y Render con sus proyectos Supabase de PRE y PROD (SCRUM-128); bloqueo por intentos fallidos (SCRUM-155).
- Aplicar las actualizaciones de parche pendientes (ArchUnit 1.5.1, JaCoCo 0.8.15) y decidir con tarjeta propia los saltos mayores (Flyway 13, Spring Boot 4).
- Sprint 4 en adelante: búsqueda de ruta con Google Maps, última milla y modo offline, y las pruebas de rendimiento y usabilidad.
- Preguntar al docente la fecha del próximo comité: no se deduce.

14. Conclusiones del proyecto

Qué aprendimos. La calidad se diseña: escribir cada métrica con su instrumento nos obligó a reconocer que varias de las primeras eran inmedibles o falsas, y bajar la vara con razones fue mejor que sostener un número que no podíamos verificar. También aprendimos que un incremento no existe hasta que está mergeado: en el Sprint 1 había trabajo hecho que ante el comité valía cero.

Qué desafíos encontramos. La primera vez con Spring Boot, Angular y mapas, el tamaño del alcance frente a dos personas con otras materias, y decisiones de infraestructura (Render, Supabase, versiones) que condicionan qué se puede prometer. La velocidad de 17 puntos del

Sprint 2 no es una capacidad demostrada, es el resultado de un sprint concentrado sobre trabajo ya adelantado.

Cómo aplicamos ingeniería de software. Arquitectura hexagonal verificada automáticamente, principios SOLID con evidencia en el código, un enfoque iterativo–incremental con Scrum y trazabilidad completa entre requisito, historia, rama, commit y pull request. Lo que sigue pendiente —despliegue, búsqueda con mapas, mediciones de rendimiento y usabilidad— está declarado, no oculto.

Referencias

International Organization for Standardization & International Electrotechnical Commission. (2011). ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. ISO/IEC.

Schwaber, K., & Sutherland, J. (2020). La guía de Scrum. <https://scrumguides.org>

Departamento Administrativo Nacional de Estadística [DANE]. Fuente de la población regional estimada (unos 285.000 habitantes) citada en la sección 1: cita exacta pendiente de confirmar por el equipo.